

Unit Test Frameworks

Assignment 8: Data-Driven Automation (DDT)

1. Problem Statement: -

Build a login script that reads multiple test cases from an external source (such as an Excel file via pandas or a local JSON/CSV file). Loop through the test data to execute combinations of correct/incorrect usernames and passwords, asserting that the proper validation errors show up for each.

2. Objective: -

To achieve Data-Driven Testing (DDT) by separating test data from test scripts, enabling a single test layout to run iteratively against multiple datasets.

3. Tools, Software, and Concepts Used: -

Python 3.x, Selenium WebDriver, Built-in CSV Module.

4. Source Code / Implementation: -

```
import csv
import time
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.chrome.options import Options

def run_ddt_test():
    chrome_options = Options()
    chrome_options.add_experimental_option("prefs", {
        "credentials_enable_service": False,
        "profile.password_manager_enabled": False
    })
    driver = webdriver.Chrome(options=chrome_options)
    driver.implicitly_wait(5)
    with open('test_data.csv', mode='r') as file:
        csv_reader = csv.DictReader(file)
        for row in csv_reader:
            driver.get("https://www.saucedemo.com/")
            driver.find_element(By.ID, "user-
name").send_keys(row['username'])
            driver.find_element(By.ID,
"password").send_keys(row['password'])
            driver.find_element(By.ID, "login-button").click()
            if row['expected_error'] != 'None':
                error_msg = driver.find_element(By.CSS_SELECTOR,
"h3[data-test='error']").text
```

```

        assert row['expected_error'] in error_msg, f"Expected
{row['expected_error']}, got {error_msg}"
        print(f"PASS: Expected error matched for
{row['username']}")
        driver.refresh()
    else:
        assert "/inventory.html" in driver.current_url, f"Login
failed for {row['username']}"
        print(f"PASS: Successful login for {row['username']}")
        driver.execute_script("window.sessionStorage.clear();")
        driver.execute_script("window.localStorage.clear();")
        driver.delete_all_cookies()
    time.sleep(2)
driver.quit()
print("All test cases executed successfully!")
run_ddt_test()

```

5. Output & Screenshots: -

```

PS D:\wipro Selenium> & C:\Users\HP\AppData\Local\Programs\Python\Python310\python.exe "d:/wipro Selenium/Ass8.py"
PASS: Successful login for standard_user
PASS: Expected error matched for locked_out_user
PASS: Expected error matched for invalid_user
All test cases executed successfully!

```

Fig.1: All test cases executed successfully

6. Result, Observation, and Conclusion: -

- Result: The script successfully read multiple credentials and expected outcomes from an external CSV file and validated them dynamically.
- Observation: Data-Driven Testing allows for extensive edge-case coverage without writing duplicate test code. Adding a new test scenario is as simple as adding a new row to the external file.
- Conclusion: Externalizing data makes tests highly flexible and easier for non-technical team members to contribute to test coverage.